

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

Оператор приведения типа

- используется для приведения типа

`operator T() const`

- может быть `explicit`
- может быть виртуальным
- не может быть `static`

explicit operator T() const

- запрещает неявное преобразование типа
- преобразования к `bool` разрешены в некоторых контекстах

Преобразования к `bool` разрешены

- инструкции

```
if (t) {}
```

```
for(; t; ) {}
```

```
while(t) {}
```

```
do { } while (t)
```

Преобразования к `bool` разрешены

- выражения

`!t`

`t && t2`

`t || t2`

`t ? "true" : "false"`

Преобразования к `bool` разрешены

- выражения (compile-time tests)

`static_assert (t) ;`

`noexcept (t)`

`explicit (t)`

`if constexpr (t)`

Идея проху плодотворна

- решает проблему доступа к битам

```
BiA Bitset::operator[] (const int32_t i)
```

```
operator BiA::bool() const noexcept
```

- но может добавить вопросов в связи с временем жизни и преобразованиями типов

Чем «плохи» цепочки?

```
std::string s = root + "/data/" +  
    folder + "/" + name;
```

- много промежуточных объектов
- много выделений памяти
- много копирования данных

Как «цепочки» облегчить?

- смириться с неизбежным
- стать ленивым
- делать работу тогда, когда это необходимо
- а как именно технически?

С сохранением синтаксиса

- использовать промежуточные «легкие» `proху`, которые помнят «что делать»
- вычислять в терминах `proху`
- выделение/копирование/обработку сделать один раз «в конце»

Почему важно про это знать?

- используется в «быстрых» библиотеках
— OpenCV, Eigen и т.п.
- может приводить к нетривиальным сообщениям об ошибках
- может потребовать модификация для отладки
- а вдруг вам такое поддерживать или писать?

С изменением синтаксиса

- использование специализированных типов (в духе `StringBuilder`)
- в `std` наиболее близкими являются строковые потоки

```
std::string s = (ostrm << root << "/data/" <<  
    folder << "/" << name) .str() ;
```

ваши вопросы

«Исчерпаемые» ресурсы

- память (утечка памяти)
- файловые дескрипторы
- потоки и примитивы синхронизации (мьютекс, семафор и т.д.)
- соединения (сетевые, БД и т.д.)
- ...

Контроль памяти

(много точек выхода – беда)

```
// внутри функции
T* p = new T;
// ...
if (condition) {
    delete p;
    return FAILURE;
}
// ...
delete p;
return SUCCESS;
```


Контроль памяти

(C-style – одна точка выхода с goto)

```
// внутри функции
T* p = new T;
// ...
if (condition) {
    res = FAILURE;
    goto cleanup;
}
// ...
cleanup:
delete p;
return res;
```

Контроль памяти

(«нетипичные циклы» - — одна точка выхода)

```
T* p = new T;  
do {  
    // ...  
    if (condition) {  
        res = FAILURE;  
        break;  
    }  
    // ...  
} while (0);  
delete p;  
return res;
```

Контроль ресурсов

(итога)

- лучше избегать `goto`-заменяющие конструкции
- помнить об исключениях
 - перехватывать и обрабатывать в проверки утомительно
- довериться RAII
 - стековые экземпляры (автоматическая память)

Как избежать «утечек»?

использовать **RAII**

(англ. Resource Acquisition Is Initialization)

получение ресурса есть инициализация

доступ к ресурсу через ответственный
класс-обертку

RAII на практике

- ресурс захватывается в конструкторе
- ресурс освобождается в деструкторе
- копирование
 - запрещено (монопольное владение)
 - «глубокое» (захват дополнительного ресурса)
 - с подсчетом ссылок (совместное владение)

Управление ресурсами

- управление = владение
 - ответственность за время жизни
 - препятствие «провокациям»
(защита «от неправильного использования»)

Как сделать универсально?

- копирование/присваивание
- доступ (к членам через)

$(*p) . x$

$p \rightarrow x$

- защиту от утечки адреса

Макет указателя

```
class Ptr {  
    public:  
        ~Ptr();  
        Ptr(const Ptr& pt);  
        Ptr& operator=(const Ptr& pt);  
    private:  
        T* d_ = nullptr;  
};
```


Управление (жизнью ресурса)

```
Ptr(const Ptr& pt)
: d_(new T(pt.d_)) {
}
```

```
~Ptr() {
    delete d_;
}
```

Копирование (глубокое ресурса)

```
Ptr& operator=(const Ptr& pt) {  
    if (this != &pt) {  
        delete d_;  
        d_ = new T(pt.d_);  
    }  
    return *this;  
}
```

Перегрузка доступа (operator*)

```
T& operator* () {  
    return *d_;  
}
```

```
const T& operator* () const {  
    return *d_;  
}
```

Перегрузка доступа (operator->)

```
T* operator->() {  
    return d_;  
}
```

```
const T* operator->() const {  
    return d_;  
}
```

Почему безопасно (drill-down behavior)

ВЫЗОВ

$p \rightarrow x$

ЭКВИВАЛЕНТЕН ВЫЗОВУ

$(p.operator \rightarrow ()) \rightarrow x$

причем детализация производится «до упора»

ваши вопросы

Монопольное владение ресурсом

- `std::unique_ptr`
- копирование запрещено
- разрешено перемещение

примеры

— классы ввода/вывода

Монопольное владение ресурсом

- объект разрушается указателем
- право владения «передается»

```
auto p1 = std::make_unique<Vec3>(1, 2, 1);  
p2 = std::move(p1);
```


Совместное владение объектом

- семантика подсчета ссылок
- `std::shared_ptr<T>`
- объект разрушается вместе с последним владельцем (подсчет ссылок)
- при присваивании и копировании

Совместное владение объектом

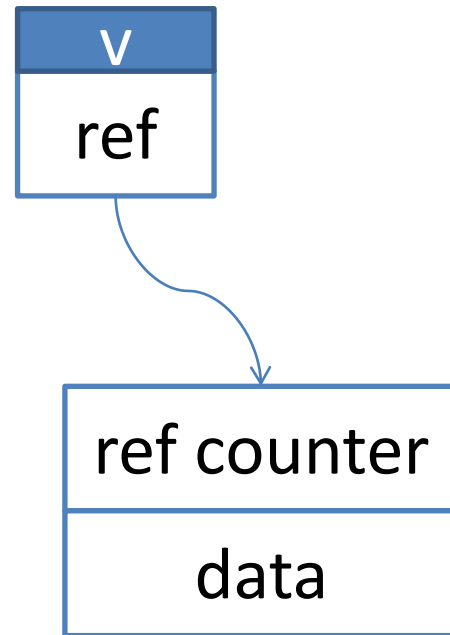
изучаем `<memory>`

- `std::make_shared<>()`
- `std::enable_shared_from_this<>`

Что такое
подсчет ссылок?

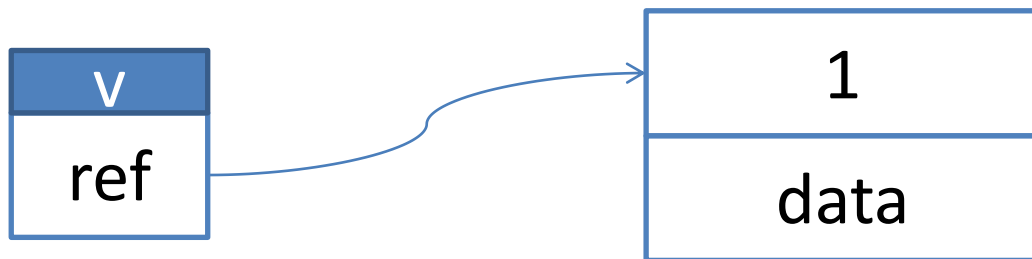
Тип с подсчетом ссылок

- совместное владение ресурсом
- при помощи подсчета ссылок
- хранятся в «куче» (heap)
 - данные
 - служебная информация



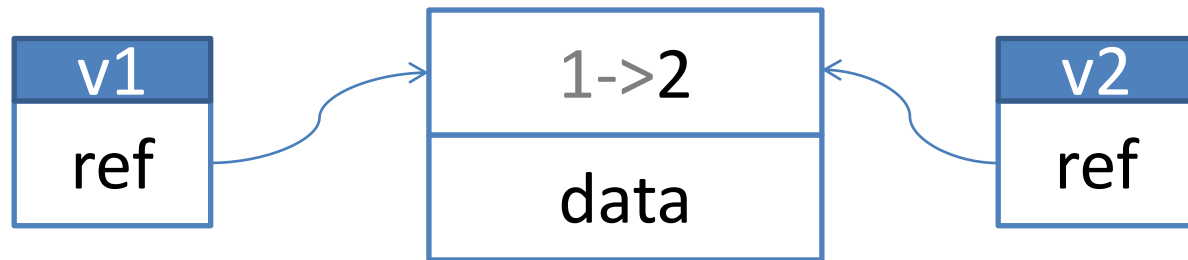
Создание (подсчет ссылок)

- при создании
 - объект (ссылка) создается (стек или куча)
 - данные (+ счетчик) создается в куче
 - счетчик ссылок устанавливается в 1



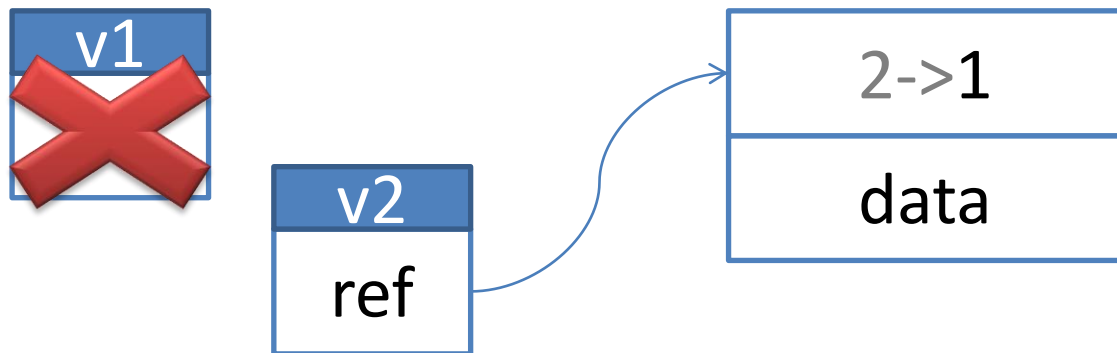
Копирование (подсчет ссылок)

- при копировании ($v2 = v1$)
 - объект (ссылка) создается (стек или куча)
 - копируется ссылка
 - увеличивается счетчик ссылок



Уничтожение (подсчет ссылок)

- при уничтожении
 - объект (ссылка) уничтожается
 - уменьшается счетчик ссылок



А что будет в конце?

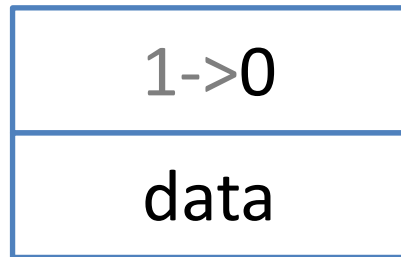
Уничтожение (без сборщика мусора)

- последний владелец освобождает ресурс



Уничтожение (сборщик мусора)

- освобождение памяти от неиспользуемых экземпляров осуществляется автоматически
- влияние пользователя ограничено



Сборщик мусора

- + упрощает написание простого кода
- потребляет дополнительные ресурсы
- работает «самостоятельно»
- усложняет написание эффективного кода

`std::shared_ptr` (детали реализации)

- объект
- управляющий блок
 - счетчик ссылок (атомарный)
 - слабый счетчик (управляет жизнью блока)
 - другие данные
(пользовательский удалитель, распределитель и т.д.)

std::make_shared

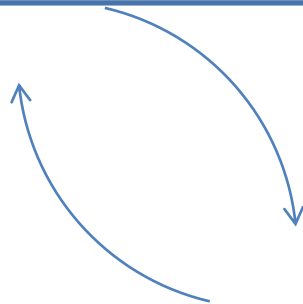
- выделяет всю необходимую память
- не оставляет места исключениям

примеры

```
std::shared_ptr<T> p1(new T)  
auto p2 = std::make_shared<T>();
```

Циклические ссылки

```
std::shared_ptr<T> next_;
```



```
std::shared_ptr<T> next_;
```

Как разорвать циклы?

`std::weak_ptr<T>`

- дополняет `shared_ptr`
- не может использоваться как указатель
- должен быть «превращен» в `shared_ptr`
 - `std::weak_ptr::lock()`
 - `shared_ptr(std::weak_ptr<T>)`

Рекомендации

- выбирайте типы в соответствии с семантикой
- используйте `make`-функции для создания
- изучите особенности управления массивами
- перемещайте умные указатели, если это возможно

ваши вопросы

std::swap B C++ 98

```
template <typename T>
void swap(T& x, T& y) {
    T tmp = x; // copy ctor
    x = y;      // assign
    y = tmp;    // assign
}
```

- raw-указатели обмениваются эффективно
- умные указатели обмениваются «тяжело»
- в C++11 язык расширили – move-semantic

lvalue и rvalue

- lvalue – выражение адресуемое (location)
- rvalue – выражение не lvalue
 - временный объект
 - литерал и т.д.

пример:

$x + 1$

&& - rvalue ссылка

- материализует и связывает имя с rvalue
- связывается только с rvalue
- является lvalue

пример:

```
int a = 10;
```

```
int& re = a;
```

```
int&& rr = a + 1;
```

Примеры

```
int    v = 1;
```

```
int&& rr = v + 5;
```

```
int&& er = v;    // compile error
```

// правая ссылка задает адрес и имя

```
int&& ee = rr;    // compile error
```

```
int&  lr = rr;    // ok
```

const lvalue reference

- продляет жизнь временны объектов

```
const int& cr = v + 8;
```

- неконстантная левая ссылка не «крепится» к rvalue

```
int v = 0;
```

```
int& er = v + 3; // compile error
```


std::move

- является преобразование типа (по сути)
- названо неудачно

пример:

```
int v = 0;
```

```
int&& rr = std::move(v);
```

Зачем всё это?

- на уровне синтаксиса правыми ссылками отмечать то, «что не жалко» (скоро само пропадет)
- использовать для оптимизации

Методы над rvalues

- метод может быть вызван для rvalue-expression

```
struct Data {           Data v;  
    int d = 0;  
    int& data() {       int& rr = v.data();  
        return d;  
    }                  // что «тут не так»?  
};                     int& er = Data{}.data();
```

Аннотация методов

- спецификатор, который позволяет перегружать метод по возможности вызова для lvalue и rvalue

пример:

T do () &; // для lvalue

T do () &&; // для rvalue

Аннотация методов (пример)

```
struct Data {  
    int d = 0;  
    int& data() & { return d; }  
};  
  
Data v;  
  
int& rr = v.data();           // ok  
  
int& er = Data{}.data();      // compile error
```

Возврат ссылок

```
int& f0(int& x) { return x; }
```

```
const int& f1(const int& x) { return x; }
```

```
int&& f2(int&& x) { return std::move(x); }
```

```
int& f3(int&& x) { return x; }
```

Возврат ссылок

```
// safe
```

```
int& f0(int& x) { return x; }
```

```
// casino
```

```
const int& f1(const int& x) { return x; }
```

```
// troubles
```

```
int&& f2(int&& x) { return std::move(x); }
```

```
// casino
```

```
int& f3(int&& x) { return x; }
```

ВАШИ ВОПРОСЫ

Перемещающий ctor и operator=

- принимает rvalue ref
- «присваивает» данные источника
- копирует крайне эффективно, по сравнению с обычным (глубоким) копированием

```
T (T&& src)
```

```
T& operator= (T&& rhs)
```

Перемещенный объект

- обычно, уже был близок к разрушению
- должен оставаться в валидном состоянии
 - объекту можно присвоить новое значение
- может быть в неопределенном состоянии
 - вызывать методы можно
 - результаты могут быть любыми
(нельзя опираться на ожидаемые результат)

Перемещающий ctor (пример)

```
Ptr(const Ptr& pt)
: d_(new T(pt.d_)) {
}
```

```
Ptr(Ptr&& pt) noexcept
: d_(pt.d_) {
    pt.d_ = nullptr;
}
```

Перемещающий operator=

(пример)

```
Ptr& operator=(Ptr&& rhs) noexcept {  
    if (this != &rhs) {  
        delete d_;  
        d_ = pt.d_;  
        pt.d_ = nullptr;  
    }  
    return *this;  
}
```

Перемещающий operator=

(пример)

```
Ptr& operator=(Ptr&& rhs) noexcept {  
    if (this != &rhs) {  
        std::swap(d_, pt.d_);  
    }  
    return *this;  
}
```

`std::swap` в C++11 и далее

```
template <typename T>
void swap(T& x, T& y) {
    T tmp = std::move(x); // move ctor
    x = std::move(y);      // move assign
    y = std::move(tmp);    // move assign
}
```

Важные детали перемещения

- перемещение по умолчанию перемещает все поля
- как работает `std::swap`, если у типа нет перемещающих версий?
- просто осуществляет копирование

Проблема implicit move

- сложные правила генерации неявных методов
- перемещение по умолчанию перемещает по умолчанию все поля класса
- если у типа нет перемещения, то происходит копирование (например, указатели)

Правило пяти

(rule of five)

если в классе есть хотя бы один, то необходимо реализовать все пять:

- конструктор копирования;
- конструктор перемещения;
- оператор копирования;
- оператор перемещения;
- деструктор.

Правило нуля

(rule of zero)

- если хотя бы один из неявных методов определен явно, все пять необходимо явно определить (RAII-тип) и не должно быть никаких других методов (соблюдение Single Responsibility Principle, SRP)
- в курсе всегда определяем все пять (default вам в помощь)

Как правильно написать перемещение

- всегда явно показываем намерение
(не опускаем объявлений неявных методов)
- используем перемещаемые типы для полей
- сложные поля перемещаем «руками» при необходимости (через `std::swap` или `std::move`)

Как правильно написать перемещение

- пробрасовать `rvalue`, где необходимо

пример:

```
Derived(Derived&& src)
    : Base(std::move(src)) {
    ...
}
```

Как правильно использовать перемещение

- `std::move` внутри `return` – подозрительно
 - риски провисания
 - поломка Return Value Optimization, RVO

пример:

```
T f() {  
    T res;  
  
    ...  
  
    return std::move(res); // trouble
```

ВАШИ ВОПРОСЫ

`std::unique_ptr<>`

- `<memory>`
- монопольное владение ресурсом
- перемещаемый, но не копируемый
(владение можно передать)

Создание `unique_ptr<T>`

- две формы

```
std::unique_ptr<T> p(new T);
```

```
auto p = std::make_unique<T>();
```

- предпочитайте `make_unique`
(безопаснее при исключениях)

Создание массива

- используйте специализацию для массива

```
auto ar = std::make_unique<T[]>(10);
```

- иначе рискуете сделать ошибку

```
std::unique_ptr<T[]> ar(new T[10]());
```

scoped_ptr

const std::unique_ptr

- идеально подходит
- не может быть скопирован
- не может быть перемещен

Где могут прятаться неприятности?

- вспоминаем очередь или стек на списке
- переделываем Node на `std::unique_ptr`
- переделываем `head_` на `std::unique_ptr`
- отлаживаем
- можем получить неприятности – где?

ВАШИ ВОПРОСЫ